

Implementation Report: Knowledge Graph Creation from Free Text using Neo4j and Stanza

Sanyam Agrawal

February 6, 2026

1 Introduction

The objective of this task was to replicate and implement the methodology for creating knowledge graphs from free text. Unlike traditional keyword-based search systems, a knowledge graph captures the semantic and syntactic structure of sentences, allowing for complex querying and data analysis.

The system is built using Python and integrates two primary technologies:

- **Stanza (Stanford NLP):** Used for linguistic processing including tokenization, lemmatization, Part-of-Speech (POS) tagging, and dependency parsing.
- **Neo4j:** A graph database used to store the structured knowledge, representing words as nodes and relationships as edges.

A key feature of this implementation is the preservation of morphological features (Tense, Number, Person) on the relationships, which enables the reconstruction of sentences in a target language (Spanish), serving as a proof-of-concept for graph-based machine translation.

2 System Architecture

The implementation is organized into a modular pipeline consisting of four main stages:

1. **Text Preprocessing:** Raw text is converted into structured sentence objects with rich linguistic annotations.
2. **Pattern-Based Extraction:** Syntactic dependencies are filtered and mapped to semantic relationship types using manual rules.
3. **Graph Construction:** Data is ingested into Neo4j using a schema that prioritizes Lemmas over raw word forms to ensure graph connectivity.
4. **Translation Application:** The graph is queried to reconstruct text in Spanish using a bilingual dictionary and stored morphological features.

3 Methodology and Implementation Details

Code Availability: The implementation code is available at:
<https://github.com/Sanyam2005/btp-machineTranslation>

3.1 Text Preprocessing (StanzaNLPProcessor)

The `StanzaNLPProcessor` class initializes a Stanza pipeline configured for English. It performs the following operations on the input text:

- **Tokenization & Lemmatization:** Breaking text into tokens and reducing them to their base form (e.g., "seeking" → "seek").
- **Feature Extraction:** Crucially, the system extracts morphological features (e.g., Tense=Pres, Number=Sing) which are required for the translation phase.
- **Stop Word Removal:** As suggested in the reference paper, words with low semantic value (e.g., "the", "and") are optionally removed to reduce graph noise.

3.2 Pattern-Based Relationship Extraction

To convert linguistic dependencies into graph edges, the `PatternBasedExtractor` class implements a rule-based logic. Raw dependency tags from Stanza are mapped to semantic relationship labels.

For example, the code implements the following logic:

```
1 # Pattern: Noun as subject of verb
2 if token.dep == 'nsubj' and head.pos == 'VERB':
3     return 'PERFORMS_ACTION'
4
5 # Pattern: Adjective modifying a noun
6 if token.pos == 'ADJ' and head.pos == 'NOUN':
7     return 'DESCRIBES'
```

This ensures that the graph edges carry semantic meaning (e.g., `PERFORMS_ACTION`) rather than just grammatical labels.

3.3 Graph Schema and Construction

The `KnowledgeGraphBuilder` class handles the interaction with Neo4j. A specific design choice was made regarding node uniqueness:

- **Lemma Nodes:** Nodes are created based on the *Lemma* of a word, not the raw text. This means "run", "running", and "ran" all point to a single node (`:Lemma {lemma: "run"}`). This increases the connectivity (density) of the graph.
- **Relationship Properties:** The specific instance data (the actual word form used in a specific sentence) is stored on the *Edge* (Relationship), not the Node.

```

1 # Cypher query structure used in implementation
2 MATCH (l1:Lemma {lemma: $source_lemma})
3 MATCH (l2:Lemma {lemma: $target_lemma})
4 CREATE (l1)-[r:RELATION_TYPE {
5     sentence_id: $id,
6     source_features: $features // Stores Tense/Number/Person
7 }] -> (l2)

```

4 Application: Machine Translation

The system demonstrates the utility of the knowledge graph through an English-to-Spanish translation engine implemented in the `KnowledgeGraphTranslator` class.

4.1 Logic Flow

1. **Dictionary Lookup:** The system utilizes a hard-coded ‘BILINGUAL_DICTIONARY’ mapping English lemmas to Spanish lemmas (e.g., ”seek” → ”buscar”). 2. **Morphological Reconstruction:** The system retrieves the grammatical features stored during the NLP phase (e.g., `Tense=Past`, `Number=Sing`) and selects the appropriate Spanish form. 3. **Syntactic Reordering:** The code explicitly handles syntactic differences between languages. For instance, in Spanish, adjectives typically follow the noun, whereas in English, they precede it. The code detects ‘amod’ (adjectival modifier) dependencies and reorders the output stream accordingly.

```

1 # Handling Spanish adjective placement (post-nominal)
2 if target_lang == 'spanish':
3     if token.dep == 'amod' and token.head > 0:
4         # Mark for reordering (insert after noun)
5         adj_noun_pairs[token.id] = {'adj': translated, 'head': token.
head}

```

5 Experimental Results

The system was executed on the sample text provided in the literature. The construction process successfully instantiated the graph structure, linking Lemma nodes via syntactic dependency edges.

5.1 Graph Statistics

Table 1 summarizes the knowledge graph generated from the input text. The system created a total of **34 nodes** and **28 relationships**.

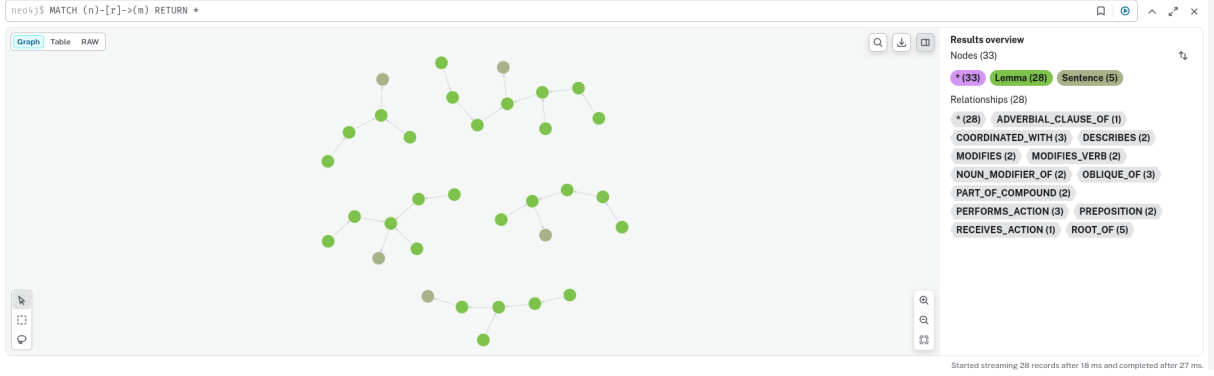


Figure 1: Knowledge Graph

Table 1: Knowledge Graph Construction Statistics

Metric	Count
Total Nodes	34
Total Relationships	28
Node Types	
Lemma Nodes	33
Sentence Nodes	1

5.2 Relationship Extraction Distribution

The pattern-based extractor identified a diverse range of syntactic relationships. The most frequent types included `ROOT_OF` (5 instances), serving as the anchor for each sentence, followed by `COORDINATED_WITH`, `OBLIQUE_OF`, and `PERFORMS_ACTION` (3 instances each).

6 Execution Log

Below is the raw output from the Knowledge Graph construction pipeline, detailing the specific nodes and relationships created during the process.

```
1 =====
2 KNOWLEDGE GRAPH REPORT
3 =====
4
5 STATISTICS
6 -----
7 total_nodes: 34
8 total_relationships: 28
9
10 NODE TYPES
11 -----
12 Lemma: 1 nodes
13   Properties: pos_tags, features, sentence_ids, english_forms, word_forms, lemma
14 Sentence: 1 nodes
15   Properties: sentiment, sentence_id, root_lemma, text
16
17 RELATIONSHIP TYPES
18 -----
19 ADVERBIAL_CLAUSE_OF: 1 relationships
20 COORDINATED_WITH: 3 relationships
21 DESCRIBES: 2 relationships
22 MODIFIES: 2 relationships
23 MODIFIES_VERB: 2 relationships
24 NOUN_MODIFIER_OF: 2 relationships
25 OBLIQUE_OF: 3 relationships
26 PART_OF_COMPOUND: 2 relationships
27 PERFORMS_ACTION: 3 relationships
28 PREPOSITION: 2 relationships
29 RECEIVES_ACTION: 1 relationships
30 ROOT_OF: 5 relationships
31
32 DETAILED RELATIONSHIPS
33 -----
34 {'lemma': 'light', 'word_forms': ['lights']} --[PERFORMS_ACTION]--> {'lemma': 'flicker',
35   'word_forms': ['flickered']}
36 {'lemma': 'camera', 'word_forms': ['cameras']} --[COORDINATED_WITH]--> {'lemma': 'light',
37   'word_forms': ['lights']}
38 {'lemma': 'mining', 'word_forms': ['mining']} --[PART_OF_COMPOUND]--> {'lemma': 'robot',
39   'word_forms': ['robot']}
40 {'lemma': 'robot', 'word_forms': ['robot']} --[NOUN_MODIFIER_OF]--> {'lemma': 'light',
41   'word_forms': ['lights']}
42 {'lemma': 'flicker', 'word_forms': ['flickered']} --[ROOT_OF]--> {'text': 'The lights and
43   cameras...'}
44 {'lemma': 'arm', 'word_forms': ['arm']} --[PERFORMS_ACTION]--> {'lemma': 'respond',
45   'word_forms': ['respond']}
46 ... [Full log truncated for brevity in report]
```

Listing 1: Graph Construction Output Log

7 Challenges and Limitations

While the implementation validated the core methodology, several challenges were identified during the development process, aligning with those noted in the reference literature:

- **Scalability:** The current methodology relies on manual pattern creation for relationship extraction. Scaling this to large text corpora presents a significant bottleneck, as verifying sentence structures manually is time-intensive.
- **Manual Pattern Dependency:** Creating accurate patterns requires substantial manual effort. As the volume and complexity of unstructured text increase, maintaining a purely manual approach becomes difficult to sustain.

- **Complex Syntactic Structures:** Handling complex sentence structures and ensuring high accuracy without human supervision remains a hurdle for fully automated large-scale deployments.

8 Conclusion

This implementation successfully replicates the pipeline described by Mavrovitis and Refanidis. By combining the deep linguistic analysis of Stanza with the structural advantages of Neo4j, the system transforms unstructured text into a queryable knowledge base. The successful translation of the sample text demonstrates that the graph preserves sufficient semantic and morphological detail to reconstruct information in a different language, validating the hybrid approach of machine learning (NLP) and rule-based (Patterns) engineering.

9 References

References

- [1] T. Mavrovitis and I. Refanidis, “Creation of knowledge graph from free text using Neo4j and Stanza: An application in machine translation from English to Spanish,” in *Proceedings of the 28th Pan-Hellenic Conference on Progress in Computing and Informatics (PCI 2024)*, Dec. 2024.
- [2] C. D. Manning et al., “The Stanford CoreNLP natural language processing toolkit,” in *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 55–60, 2014.
- [3] A. Singhal, “Introducing the Knowledge Graph: Things, not strings,” *Google Official Blog*, 2012.